

Fake News Detection Using LSTM Networks and Pre-trained BERT Models

Esha Hooda | Student ID: 014796990

Tyler Biesemeyer | Student ID: 016488979

Tarif Khan | Student ID: 017050683

Course: CS 177 Machine Learning

GitHub: <https://github.com/TarifEKhan/Fake-news-detection>

Date: May 11, 2025

1. Introduction

Background

The rapid spread of misinformation through online news platforms and social media has become a critical societal problem. Fake news shapes public opinion, influences political decisions, and undermines trust in institutions. High-profile cases surrounding elections and public health crises demonstrate the urgency of developing automated detection systems that operate at scale.

Natural Language Processing (NLP) and deep learning provide promising solutions. By training models on large labeled corpora of news articles, classifiers can learn to distinguish real from fabricated content, supporting fact-checkers, content moderators, and journalists.

Objectives

- Build and evaluate a custom Bidirectional LSTM (BiLSTM) classifier.
- Fine-tune a pre-trained BERT transformer model on the same task.
- Establish performance baselines with classical ML models (SVM, Random Forest).
- Compare all four models across Accuracy, Precision, Recall, and F1-Score.
- Analyze the accuracy-vs-efficiency tradeoff between simple and complex models.

Significance

Automated fake news detection can assist fact-checking pipelines and content moderation systems at scale. This project contributes by demonstrating that both lightweight (LSTM) and heavyweight (BERT) models achieve strong performance, with the choice depending on available computational resources.

2. Literature Review

Traditional Machine Learning

Early approaches relied on hand-crafted features combined with classical classifiers. Naive Bayes and Support Vector Machines (SVMs) using TF-IDF representations achieved reasonable accuracy on early benchmarks. Castillo et al. (2011) showed that credibility signals in text and metadata could be exploited for fake news identification. However, these methods fail to capture long-range semantic dependencies in text.

Deep Learning Approaches

Recurrent Neural Networks and their gated variants, LSTMs and GRUs, improved performance significantly by modeling sequential context. Wang (2017) introduced the LIAR benchmark and showed LSTM-based models outperform SVMs on multi-class fake news classification. Bidirectional LSTMs further improved results by capturing context from both directions. CNNs and hybrid CNN-LSTM architectures have also proven effective by capturing local n-gram features alongside sequential patterns.

Transformer-Based Models

The introduction of BERT (Devlin et al., 2019) marked a paradigm shift in NLP. Pre-trained on 3.3 billion words using masked language modeling and next-sentence prediction, BERT achieves state-of-the-art performance when fine-tuned on downstream tasks. Kula et al. (2020) demonstrated that fine-tuned BERT models consistently outperform LSTM-based approaches on fake news datasets, though at significantly higher computational cost.

The Accuracy-Efficiency Tradeoff

A recurring theme in the literature is the tradeoff between model complexity and performance. While BERT achieves superior accuracy, it requires orders of magnitude more parameters and compute than LSTMs. In resource-constrained settings such as mobile or edge computing, simpler models remain relevant. This project directly addresses this tradeoff using the same dataset for all models to enable fair comparison.

3. Methodology

Dataset

We use the Fake and Real News Dataset (Bisaillon, 2020) from Kaggle, consisting of two CSV files: Fake.csv and True.csv. Real news articles are primarily sourced from Reuters, while fake news articles are sourced from websites flagged by PolitiFact and other fact-checking organizations.

Property	Value
Total Articles	44,898
Fake Articles	23,481 (52.3%)
Real Articles	21,417 (47.7%)
Features	title, text, subject, date
Label	0 = Fake, 1 = Real
Working Subset	20,000 (10,000 Fake + 10,000 Real, balanced)
Source	Kaggle: clmentbisaillon/fake-and-real-news-dataset

Preprocessing Pipeline

The following steps were applied in order to prepare raw text for modeling:

Combine title + body text: Title carries strong discriminative signal, especially sensationalism in fake news.

Lowercase normalization: Reduces vocabulary size; "Trump" and "trump" represent the same token.

URL removal: HTTP links add noise with no semantic value.

HTML tag removal: Removes residual markup from web scraping.

Punctuation and number removal: Focuses the model on word content rather than formatting.

NLTK word tokenization: Splits text into individual word tokens.

Stopword removal: Removes common words such as "the" and "is" that carry no discriminative signal.

Short token filtering: Removes tokens of 2 characters or fewer, which are rarely meaningful.

Sequence encoding for LSTM: Keras Tokenizer with vocab=60,000, padded to length 400.

WordPiece encoding for BERT: BertTokenizer with max_length=256, padding and truncation enabled.

Neural Network Models

Bidirectional LSTM (Custom Model)

The BiLSTM model uses an Embedding layer to project vocabulary indices into 128-dimensional dense vectors. Two stacked Bidirectional LSTM layers (128 and 64 units) capture sequential dependencies in both the forward and backward directions. Dropout layers (rate=0.3) after each LSTM and before the Dense layer regularize the model against overfitting. The output layer uses sigmoid activation for binary classification.

BERT (Pre-trained Model)

We fine-tune bert-base-uncased from the HuggingFace Transformers library. BERT uses a 12-layer Transformer encoder with 768 hidden dimensions and 12 attention heads, totaling approximately 110 million parameters. A linear classification head is added on top of the [CLS] token representation and all layers are fine-tuned end-to-end on our labeled data.

4. Implementation

Tools and Frameworks

Component	Tool / Library
Language	Python 3.10
LSTM	TensorFlow 2.20 / Keras
BERT	PyTorch 2.10 + HuggingFace Transformers
Baseline Models	scikit-learn (LinearSVC, RandomForestClassifier)
Preprocessing	NLTK (tokenization, stopwords)
Data Loading	kagglehub, pandas
Visualization	matplotlib, seaborn, wordcloud
Environment	Google Colab (T4 GPU for BERT, CPU for LSTM)

BiLSTM Architecture

```
Sequential([Embedding(60000, 128, input_length=400), Bidirectional(LSTM(128,
return_sequences=True)), Dropout(0.3), Bidirectional(LSTM(64)), Dropout(0.3),
Dense(64, activation="relu"), Dropout(0.3), Dense(1, activation="sigmoid")])
```

Layer-by-Layer Summary

Layer	Output Shape	Purpose
Embedding	(400, 128)	Maps word IDs to dense vectors
BiLSTM (128 units)	(400, 256)	Bidirectional sequence context
Dropout (0.3)	(400, 256)	Regularization
BiLSTM (64 units)	(128,)	Higher-level abstraction
Dropout (0.3)	(128,)	Regularization
Dense (64, ReLU)	(64,)	Nonlinear feature combination
Dense (1, Sigmoid)	(1,)	Binary classification output

BERT Fine-tuning Setup

```
bert_model = BertForSequenceClassification.from_pretrained("bert-base-uncased",
num_labels=2) optimizer = AdamW(bert_model.parameters(), lr=2e-5,
weight_decay=0.01) scheduler = get_linear_schedule_with_warmup(optimizer,
num_warmup_steps=150, num_training_steps=1500)
```

5. Experimental Setup

Data Splits

We sampled 20,000 articles (10,000 fake, 10,000 real) for a balanced and computationally efficient working set, then applied a stratified 80/10/10 split. Stratification ensures both classes are equally represented in each partition, preventing evaluation bias.

Split	Size	Fake	Real	Purpose
Training	16,000	8,000	8,000	Fit model parameters
Validation	2,000	1,000	1,000	Monitor overfitting
Test	2,000	1,000	1,000	Final unbiased evaluation

Note: Due to GPU memory constraints, BERT was trained on a subset of 8,000 training samples, 1,000 validation samples, and 1,000 test samples. All other models used the full 16,000 / 2,000 / 2,000 split.

Hyperparameters

Parameter	BiLSTM	BERT
Model	Custom BiLSTM	bert-base-uncased
Vocabulary Size	60,000	WordPiece (~30,522)
Sequence Length	400 tokens	256 tokens
Embedding Dim	128	768 (pre-trained)
Batch Size	64	16
Learning Rate	1e-3	2e-5
Optimizer	Adam	AdamW (wd=0.01)
Epochs	10 max (EarlyStopping)	3
Dropout	0.3	0.1 (default)
LR Scheduler	ReduceLROnPlateau	Linear warmup (10%)
Early Stopping	patience=3	N/A
Actual Epochs Run	6 (best at epoch 3)	3

Evaluation Metrics

All models are evaluated using Accuracy, Precision, Recall, and F1-Score. F1-Score is our primary metric because it balances precision and recall. In fake news detection, both false positives (flagging real news as fake) and false negatives (missing fake news) carry real-world consequences, so neither can be ignored in isolation.

6. Results

Performance Summary

Table 5 presents the performance of all four models on their respective held-out test sets. All models achieve strong performance, consistent with published benchmarks on this dataset.

Model	Accuracy	Precision	Recall	F1-Score
SVM (TF-IDF)	0.9950	0.9920	0.9980	0.9950
Random Forest	0.9945	0.9940	0.9950	0.9945
BiLSTM (custom)	0.9965	0.9980	0.9950	0.9965
BERT (fine-tuned)	0.9990	0.9990	0.9990	0.9990

Table 5. Test set performance. BERT was evaluated on a 1,000-sample subset; SVM, RF, and BiLSTM on the full 2,000-sample test set.

Confusion Matrix Breakdown

Model	TN (Fake OK)	FP (Fake->Real)	FN (Real->Fake)	TP (Real OK)
SVM	990	10	0	1000
Random Forest	990	10	1	999
BiLSTM	995	5	2	998
BERT	520	0	1	479

Table 6. Raw confusion matrix values. BERT test set = 1,000 samples; others = 2,000 samples.

BiLSTM Training History

Epoch	Train Acc	Val Acc	Train Loss	Val Loss
1	0.9779	0.9965	0.0528	0.0229
2	0.9994	0.9970	0.0024	0.0245
3*	0.9996	0.9970	0.0019	0.0227
4	0.9998	0.9970	0.0020	0.0242
5	0.9998	0.9970	0.00067	0.0316
6	1.0000	0.9970	0.0000436	0.0340

Table 7. BiLSTM training history. Epoch 3* = best epoch; EarlyStopping triggered at epoch 6, weights restored from epoch 3.

BERT Training History

Epoch	Train Acc	Val Acc	Train Loss	Val Loss
1	0.9515	0.9980	0.1169	0.0087

2	0.9982	0.9990	0.0087	0.0044
3	0.9994	1.0000	0.0039	0.0001

Table 8. BERT training history. Val accuracy reached 1.0000 by epoch 3.

Key Observations

BERT achieves the highest performance across all metrics, confirming the value of pre-trained language models.

BiLSTM outperforms both SVM and Random Forest, validating the value of sequential context modeling over bag-of-words features.

SVM with TF-IDF achieves 99.50% F1, demonstrating that classical ML remains competitive when class boundaries are linguistically clear.

All models exceed 99.4% accuracy, indicating strong stylistic signals separating real from fake news in this dataset.

BERT made only 1 error on 1,000 test samples, approaching near-perfect classification on this benchmark.

7. Discussion

Why BERT Outperforms LSTM

Three key factors explain BERT's superiority. First, pre-training: BERT has been trained on 3.3 billion words before seeing any labeled data, giving it rich prior knowledge of language semantics. The BiLSTM starts from random weights and learns exclusively from our 16,000 training samples. Second, self-attention: BERT's attention mechanism lets every word attend directly to every other word in a single step, capturing long-range dependencies that LSTMs struggle with despite gating. Third, WordPiece tokenization handles unseen words by decomposing them into known subwords, while our LSTM assigns all unknown words the same single OOV token.

Why SVM Is Competitive

This dataset has clear stylistic differences between classes. Fake news articles in this corpus tend to use sensationalist language, emotional appeals, informal writing, and opinion-heavy framing. Real news from Reuters follows formal journalistic conventions with neutral, fact-based language. These patterns are well-captured by TF-IDF word frequency features, allowing a linear classifier to achieve 99.5% F1 without any sequential modeling.

The Accuracy-Efficiency Tradeoff

The BiLSTM achieves 99.65% F1 compared to BERT's 99.90% F1, a gap of only 0.25%, while using approximately 8.5 million parameters versus BERT's 110 million. BERT required a GPU and ~20 minutes of training on 8,000 samples. The BiLSTM ran on CPU and required ~75 minutes on 16,000 samples. For deployment in low-resource environments, the BiLSTM represents a compelling choice given this minimal performance gap.

Comparison with Existing Literature

Our results are consistent with published benchmarks on this dataset. Aphiwongsophon and Chongstitvatana (2018) reported SVM accuracy near 97% on a similar corpus. Several BERT-based papers on this exact dataset report 98-99%+ accuracy. Our BiLSTM achieving 99.65% accuracy is competitive with deep learning baselines from the literature, validating our architecture and preprocessing choices.

Limitations

Dataset bias: Real news is predominantly from Reuters, so models may partially learn source style rather than truthfulness.

Domain generalization: Models trained on political news may not transfer to health misinformation or financial fake news.

Temporal drift: News language evolves; models may degrade on articles from different time periods.

BERT subset: Due to Colab GPU limits, BERT was trained and evaluated on a smaller subset (8,000 train / 1,000 test), which may slightly inflate metrics on an easier sample.

No metadata features: We used text only; source credibility, social sharing patterns, or author history could further improve performance.

Areas for Improvement

Use RoBERTa or DistilBERT for a better accuracy-efficiency tradeoff.

Apply cross-lingual models to detect fake news in multiple languages.

Incorporate ensemble methods combining LSTM and BERT predictions.

Add explainability tools such as LIME or SHAP to identify which words drive predictions.

Evaluate on out-of-domain datasets to test generalization beyond this benchmark.

8. Conclusion

This project successfully implemented and compared four models for fake news detection: SVM and Random Forest as classical baselines, a custom Bidirectional LSTM as a deep sequential model, and fine-tuned BERT as a state-of-the-art transformer. All models were evaluated on the same Kaggle fake news benchmark using Accuracy, Precision, Recall, and F1-Score.

BERT achieved the highest performance with 99.90% F1, making only 1 error on 1,000 test samples. The BiLSTM achieved 99.65% F1 using approximately 8% of BERT's parameters, confirming that the central thesis of this project: simpler sequential models remain viable in resource-constrained settings, with only a marginal performance penalty compared to large pre-trained models.

The comprehensive preprocessing pipeline, combining title and body text, removing noise, and applying stopwords filtering, was critical to achieving strong performance across all models. The consistency of results across model types also reveals that this dataset has strong stylistic signals, which should be considered when interpreting benchmark numbers.

Member Contributions

Task	Contributors
Project Setup and GitHub	Tarif Khan
Dataset Exploration	Tyler Biesemeyer, Esha Hooda, Tarif Khan
Data Visualization	Tyler Biesemeyer, Esha Hooda
Data Cleaning and Preprocessing	Esha Hooda, Tyler Biesemeyer, Tarif Khan
LSTM Implementation	Tarif Khan, Esha Hooda
BERT Implementation	Tyler Biesemeyer, Tarif Khan
Baseline Models (SVM, RF)	Esha Hooda, Tyler Biesemeyer
Evaluation and Visualization	All Members
Report Writing	All Members

References

- [1] Bisailon, C. (2020). *Fake and Real News Dataset*. Kaggle.
<https://www.kaggle.com/clmentbisailon/fake-and-real-news-dataset>
- [2] Devlin, J., Chang, M. W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL-HLT 2019*.
- [3] Wang, W. Y. (2017). Liar, Liar Pants on Fire: A New Benchmark Dataset for Fake News Detection. *ACL 2017*.
- [4] Castillo, C., Mendoza, M., and Poblete, B. (2011). Information Credibility on Twitter. *WWW 2011*.
- [5] Kula, S., Choras, M., and Kozik, R. (2020). Application of the BERT-based Architecture in Fake News Detection. *IFIP CISIM 2020*.
- [6] Aphiwongsophon, S. and Chongstitvatana, P. (2018). Detecting Fake News with Machine Learning Method. *ECTI-CON 2018*.
- [7] Chollet, F. (2021). *Deep Learning with Python (2nd ed.)*. Manning Publications.
- [8] Wolf, T. et al. (2020). HuggingFace Transformers: State-of-the-art Natural Language Processing. *EMNLP 2020*.
- [9] Hochreiter, S. and Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780.